ATHANASIOS GOURDOMICHALIS
Numerical Analysis Python Programs
Cholesky decomposition

This pdf file provides:

- A short intro to Cholseky decomposition
- The program's problem statement (assignment prompt and requirements)
- The code description and implementation details.

1. Introduction to Cholesky Decomposition

- **Cholesky decomposition** (or Cholesky factorization) is an efficient **direct method** used in **numerical linear algebra** to **decompose a matrix into simpler parts**. Specifically, any **real**, **symmetric**, and **positive-definite** (SPD) **matrix S** can be uniquely decomposed into the **product**:

$$S = L \cdot L^T$$

where **L is a lower triangular matrix** with **strictly positive real diagonal entries**, and **L^T is its transpose**.

- This method has **key practical** and **computational advantages**:
  - **Guaranteed Existence:** The **algorithm succeeds if and only if** the **matrix is symmetric** and **positive-definite**.
  - **No Pivoting Required:** **Unlike general LU factorization** (**Gaussian elimination**), Cholesky decomposition is inherently **numerically stable** for **SPD matrices** and **does not require row or column interchanges** (**pivoting**).
  - **Efficiency:** It **requires roughly half the floating-point operations** (approx. **$n^3/3$ flops**) of the **LU decomposition** (**$2n^3/3$ flops**). Since **no pivoting overhead is needed**, it **runs more than twice as fast in practice**.

2. Problem Statement

- **Objective:** We **must implement** the **column-oriented Cholesky decomposition algorithm** in **`Python`** using **`NumPy`** and **standard math functions**.

- **Input:** A test matrix $\boldsymbol{S}$. To **ensure the matrix is symmetric** and **positive-definite, we construct it via an auxiliary lower triangular matrix** $\boldsymbol{A}$ with **full rank** ($\boldsymbol{S} = \boldsymbol{A}^{\boldsymbol{T}} \cdot \boldsymbol{A}$).

  - **Matrix** A **(Initial Lower Triangular Matrix** with a **full rank)**

$$\boldsymbol{A} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{1} & \mathbf{2} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{1} & \mathbf{2} & \mathbf{3} & \mathbf{0} & \mathbf{0} \\ \mathbf{1} & \mathbf{2} & \mathbf{3} & \mathbf{4} & \mathbf{0} \\ \mathbf{1} & \mathbf{2} & \mathbf{3} & \mathbf{4} & \mathbf{5} \end{bmatrix}$$

  - **Transpose Matrix** A^T **(An upper Triangular Matrix)**

$$\boldsymbol{A^T} = \begin{bmatrix} \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} \\ \mathbf{0} & \mathbf{2} & \mathbf{2} & \mathbf{2} & \mathbf{2} \\ \mathbf{0} & \mathbf{0} & \mathbf{3} & \mathbf{3} & \mathbf{3} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{4} & \mathbf{4} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{5} \end{bmatrix}$$

  - **Symmetric Matrix** S **(**$\boldsymbol{S} = \boldsymbol{A^T} \cdot \boldsymbol{A}$**)**

$$\boldsymbol{S} = \begin{bmatrix} \mathbf{5} & \mathbf{8} & \mathbf{9} & \mathbf{8} & \mathbf{5} \\ \mathbf{8} & \mathbf{16} & \mathbf{18} & \mathbf{16} & \mathbf{10} \\ \mathbf{9} & \mathbf{18} & \mathbf{27} & \mathbf{24} & \mathbf{15} \\ \mathbf{8} & \mathbf{16} & \mathbf{24} & \mathbf{32} & \mathbf{20} \\ \mathbf{5} & \mathbf{10} & \mathbf{15} & \mathbf{20} & \mathbf{25} \end{bmatrix}$$

- **Requirements:**
  - We must complete **the `cholesky(a)` function** to **compute the lower triangular factor L in-place without using high-level built-in solvers** like ***`scipy.linalg.cholesky`*** or ***`np.linalg.cholesky`***.

    (The solution is **provided completed,** both in "***`cholesky-decomp.py`***" and "***`cholesky-decomp.ipynb`***")

  - **We must verify the result** by **checking whether the matrix product $L \cdot L^T$ reconstructs the input matrix S** within **numerical precision**.

3. Code Description & Implementation Details

The algorithm **executes** the **pseudocode** using **column-by-column updates, adapted** to `Python`'s **zero-based indexing**:

- **Constructing the SPD Matrix:** We define a **5×5 lower triangular matrix A** with **non-zero diagonal entries**, which ensures **full column rank**. Computing **$S = A^T \cdot A$** yields **a symmetric** and **strictly positive-definite matrix** suitable for testing.
- **The `cholesky(a)` function:**
  - **Memory Management:** The function creates **a working copy** (`temp = a.copy()`) so that the **input data is not modified directly**.
  - **Column Update Loop (`k` loop):** For **each column j**, it iterates **over all preceding columns $k < j$** and **updates the entries below** and **on the diagonal**:

    $$\texttt{temp}[i,j] \leftarrow \texttt{temp}[i,j] - (\texttt{temp}[i,k] \cdot \texttt{temp}[j,k]), \quad \texttt{for } i \geq j$$

    This removes **the contributions** of **previous columns**.

- Diagonal Entry: The **pivot element** is **updated** by **taking its square root**:

$$\texttt{temp}[j, j] \leftarrow \sqrt{\texttt{temp}[j, j]}$$

- Scaling Subdiagonal Entries: The **remaining elements** in **column** j **below the diagonal** are **scaled** by **dividing them** by **the newly updated diagonal element**:

$$\texttt{temp}[k, j] \leftarrow \frac{\texttt{temp}[k, j]}{\texttt{temp}[j, j]}, \quad \texttt{for } k > j$$

- Triangular Masking: Finally, `np.tril(temp)` clears any **unneeded values** in the **upper triangle** and **isolates the lower triangular factor** L.

- Numerical Verification: The **verification step** runs `np.dot(L, L.T)`. **Evaluating this product** prints an **output** that **numerically matches** the **entries of** S, confirming the **correctness** of the **decomposition**.